

Εργασία 2: Εύρεση επικρατέστερου στοιχείου με Count-Min

Αλγοριθμικές Τεχνικές για Δεδομένα Ευρείας Κλίμακας

ΑΜ: 4940, Φουρκιώτης Αθανάσιος

1. Εισαγωγή

Σε αυτή την εργασία θέλω να βρω στο τέλος μιας ροής από updates, ποια είναι η συμβολοσειρά των 100 bits που τελικά έχει τιμή 100. Από την εκφώνηση βλέπω ότι στο τέλος μένουν 1000 συμβολοσειρές με μη μηδενικό μετρητή. Οι 999 έχουν τιμή 10 και μόνο μία έχει τιμή 100 άρα αυτή είναι η βαριά συμβολοσειρά που ψάχνω. Το απλό Count-Min sketch θα ήταν αρκετό αν ήξερα ήδη ποιο string θέλω να ρωτήσω. Εδώ όμως δεν το ξέρω από πριν οπότε χρειάζεται να κρατήσω παραπάνω πληροφορία. Για αυτό σε κάθε bucket δεν αποθηκεύω μόνο το συνολικό άθροισμα αλλά και 100 αθροίσματα για τα bits. Έτσι αν η βαριά συμβολοσειρά απομονωθεί σε κάποιο bucket μπορώ μετά να τη φτιάξω ξανά bit προς bit.

2. Βασική ιδέα

Σε κάθε update (s, c) ενημερώνω το αντίστοιχο bucket σε κάθε γραμμή όπως στο Count-Min. Μετά για κάθε bit του s που είναι 1 προσθέτω το c και στο αντίστοιχο bit_sum του bucket. Η ιδέα είναι ότι το bucket δεν κρατάει μόνο πόσο βάρος πέρασε από εκεί αλλά και σε ποιες θέσεις εμφανίστηκαν άσσοι. Αν ένα bucket περιέχει μόνο τη βαριά συμβολοσειρά x με βάρος 100 το συνολικό άθροισμα του bucket είναι $\text{total} = 100$ αλλά και για κάθε bit j , το $\text{bit_sum}[j]$ είναι 100 αν $x_j = 1$ και 0 αν $x_j = 0$. Άρα αν βρεθεί τέτοιο bucket η ανακατασκευή δεν έχει άλλη τυχαιότητα. Απλά κοιτάω τα bit-sums και βάζω 1 όπου το άθροισμα είναι 100.

3. Αρνητικά updates

Τα αρνητικά updates δεν μου χαλάνε τη μέθοδο γιατί όλες οι πράξεις που κάνω είναι γραμμικές. Δηλαδή αν ένα string πάρει πρώτα +50 και μετά -50, τότε αφαιρείται από το total και από τα bit_sums ακριβώς ότι είχε προστεθεί. Οπότε τελικά είναι σαν να βλέπω μόνο τους τελικούς μετρητές.

4. Hash functions και δομή

Χρησιμοποιώ d γραμμές και w buckets ανά γραμμή. Κάθε 100-bit συμβολοσειρά

$$s = (s_0, s_1, \dots, s_{99})$$

θεωρείται ως διάνυσμα στο $\mathbb{Z}_p^{100}$, όπου κάθε s_j είναι 0 ή 1. Επιλέγω

$$p = 2^{127} - 1$$

που είναι πρώτος αριθμός και είναι μεγαλύτερος από 2^{100} . Για κάθε γραμμή i επιλέγω ανεξάρτητα:

$$a_i = (a_{i,0}, a_{i,1}, \dots, a_{i,99}) \text{ in } \mathbb{Z}_p^{100}$$
$$b_i \text{ in } \mathbb{Z}_p$$

Η hash function της γραμμής i είναι:

$$g_i(s) = (\sum_{j=0}^{99} a_{i,j} s_j + b_i) \bmod p$$
$$h_i(s) = g_i(s) \bmod w$$

Η οικογένεια g_i είναι ουσιαστικά η Vec_p οικογένεια με εσωτερικό γινόμενο modulo p . Για δύο διαφορετικά strings x και y υπάρχει τουλάχιστον ένα bit στο οποίο διαφέρουν. Από εκεί βγαίνει ότι το $g_i(x) - g_i(y)$ κατανέμεται ομοιόμορφα στο $\mathbb{Z}_p$ άρα πριν το τελευταίο $\bmod w$ έχουμε την κλασική pairwise-independent συμπεριφορά. Ένα μικρό σημείο είναι ότι μετά κάνω και $\bmod w$. Αυτό σημαίνει ότι η πιθανότητα σύγκρουσης δεν είναι ακριβώς $1/w$, εκτός αν το w διαιρεί το p . Τα buckets αντιστοιχούν σε λίγο άνισα πλήθη τιμών του $\mathbb{Z}_p$, δηλαδή σε $\text{floor}(p/w)$ ή $\text{ceil}(p/w)$ τιμές.

$$\Pr[h_i(x) = h_i(y)] = \sum_{r=0}^{w-1} \Pr[g_i(x) \bmod w = r]$$
$$\Pr[g_i(y) \bmod w = r]$$

και ισχύει

$$\Pr[h_i(x) = h_i(y)] \leq 1/w + w/(4p^2).$$

Για $p = 2^{127} - 1$ και $w = 1370$, ο έξτρα όρος είναι πάρα πολύ μικρός. Άρα στους υπολογισμούς χρησιμοποιώ το $1/w$ σαν πολύ καλή προσέγγιση. Δεν είναι όμως ακριβής ισότητα και αυτό οφείλεται μόνο στο τελικό $\bmod w$.

Στον κώδικα, πρακτικά κρατάω το `table[d][w]` για τα συνολικά αθροίσματα, τα `bit_sums` για τα 100 αθροίσματα ανά bucket και `touched` buckets, ώστε στο τέλος να μη γίνεται σάρωση άδειων sparse δομών. Τα `bit_sums` τα κράτησα sparse,

δηλαδή φτιάχνονται μόνο όταν κάποιο bucket χρησιμοποιηθεί. Αυτό έγινε κυρίως για να τρέχει πιο άνετα το πείραμα.

5. Επιλογή παραμέτρων

Θέλω η βαριά συμβολοσειρά να μείνει μόνη της σε τουλάχιστον μία γραμμή με πιθανότητα περίπου 99%. Αν θεωρήσω, όπως στην ανάλυση, ότι τα buckets είναι σχεδόν ομοιόμορφα, τότε σε μία γραμμή η πιθανότητα να μην πέσει κανένα από τα 999 ελαφριά strings μαζί με τη βαριά είναι:

$$q = (1 - 1/w)^{999}$$

Για $w = 1370$:

$$q = (1 - 1/1370)^{999} \approx 0.482$$

Με d ανεξάρτητες γραμμές:

$$P_{\text{success}} = 1 - (1 - q)^d$$

Για $d = 7$:

$$P_{\text{success}} = 1 - (1 - 0.482)^7 \approx 0.990$$

Άρα με $w = 1370$ και $d = 7$ παίρνω περίπου 99% πιθανότητα επιτυχίας, με βάση αυτό το μοντέλο. Για να μην φαίνεται ότι διάλεξα απλά τυχαία τις τιμές, έκανα και μια μικρή αναζήτηση για $d = 1, \dots, 100$. Για κάθε d βρήκα το μικρότερο w που δίνει $P_{\text{success}}(d, w) \geq 0.99$ και μετά κράτησα τον συνδυασμό με το μικρότερο $d \cdot w$ επειδή το $d \cdot w = 9590$ μετράει μόνο τα βασικά Count-Min buckets. Δεν είναι όλη η μνήμη της δομής γιατί σε κάθε bucket κρατάω και 100 επιπλέον bit-sums.

$$d \cdot w \cdot (1 + 100) = 7 \cdot 1370 \cdot 101 = 968590 \text{ ακέραιοι μετρητές.}$$

Στην πραγματική υλοποίηση τα bit_sums είναι sparse, οπότε δεν δεσμεύονται όλα αυτά από την αρχή. Σαν θεωρητικό μέγεθος μνήμης πρέπει να αναφερθούν και οι 100 επιπλέον μετρητές ανά bucket.

6. Γιατί δουλεύει η ανακατασκευή

Έστω x το μοναδικό string με τελικό βάρος 100. Αν σε κάποια γραμμή i κανένα από τα 999 ελαφριά strings δεν πέσει στο ίδιο bucket με το x τότε το bucket $h_i(x)$ έχει μέσα μόνο τη συνεισφορά του x . Άρα:

```
total = 100
bit_sum[j] = 100*x_j για κάθε j
```

Οπότε ο αλγόριθμος διαβάζει αυτά τα bit_sums και βρίσκει ακριβώς τα bits του x.

Γιατί δεν επιστρέφεται λάθος bucket

Στο τέλος της ροής οι μη μηδενικοί τελικοί μετρητές είναι μόνο ένα string με βάρος 100 αλλά και 999 διαφορετικά strings με βάρος 10. Ο αλγόριθμος αποδέχεται ένα bucket μόνο αν:

```
total = 100
και κάθε bit_sum[j] είναι 0 ή 100.
```

Τώρα παίρνω ένα bucket που περνάει αυτόν τον έλεγχο και βλέπω τι μπορεί να περιέχει. Αν περιέχει τη βαριά συμβολοσειρά και έστω ένα ελαφρύ string, τότε το total θα ήταν τουλάχιστον $100 + 10 = 110$, άρα δεν θα περνούσε τον έλεγχο. Επομένως, αν ένα αποδεκτό bucket περιέχει τη βαριά συμβολοσειρά την περιέχει μόνη της. Αν δεν περιέχει τη βαριά συμβολοσειρά, τότε για να έχει total = 100 πρέπει να περιέχει ακριβώς 10 ελαφριά strings, αφού το καθένα έχει βάρος 10. Για μια θέση bit j, το bit_sum[j] θα είναι:

```
10 * πλήθος_ελαφριών_strings_με_bit_1_στη_θέση_j.
```

Για να βγαίνει πάντα 0 ή 100, σε κάθε θέση j πρέπει είτε και τα 10 strings να έχουν bit 0 είτε και τα 10 strings να έχουν bit 1. Δηλαδή τα 10 strings θα ήταν ίδια σε όλες τις 100 θέσεις. Αυτό όμως δεν γίνεται αφού οι συμβολοσειρές που διάλεξα είναι διαφορετικές. Άρα ένα bucket που περνάει τον έλεγχο δεν μπορεί να δώσει λάθος απάντηση. Αν ο αλγόριθμος επιστρέψει string, αυτό είναι το σωστό heavy string. Η αποτυχία συμβαίνει μόνο όταν η βαριά συμβολοσειρά έχει σύγκρουση με κάποιο ελαφρύ string σε όλες τις γραμμές.

7. Αλγόριθμος

Σε κάθε update (s, c) αρχικά υπολογίζω τις d` hash θέσεις του s` μετά προσθέτω το c` στο total` κάθε αντίστοιχου bucket και τέλος για κάθε bit του s` που είναι 1, προσθέτω το c` στο αντίστοιχο bit_sum`. Έτσι κοιτάω τα buckets που έχουν χρησιμοποιηθεί. Αν βρω bucket με total = 100 και όλα τα bit_sums ίσα με 0 ή 100, τότε από αυτά φτιάχνω την απάντηση:

```
if total == 100 and all(v == 0 or v == 100 for v in bits):
    answer = 0
    for bit in range(100):
```

```
    if bits[bit] == 100:
        answer |= (1 << bit)
    return answer
```

8. Σημεία υλοποίησης

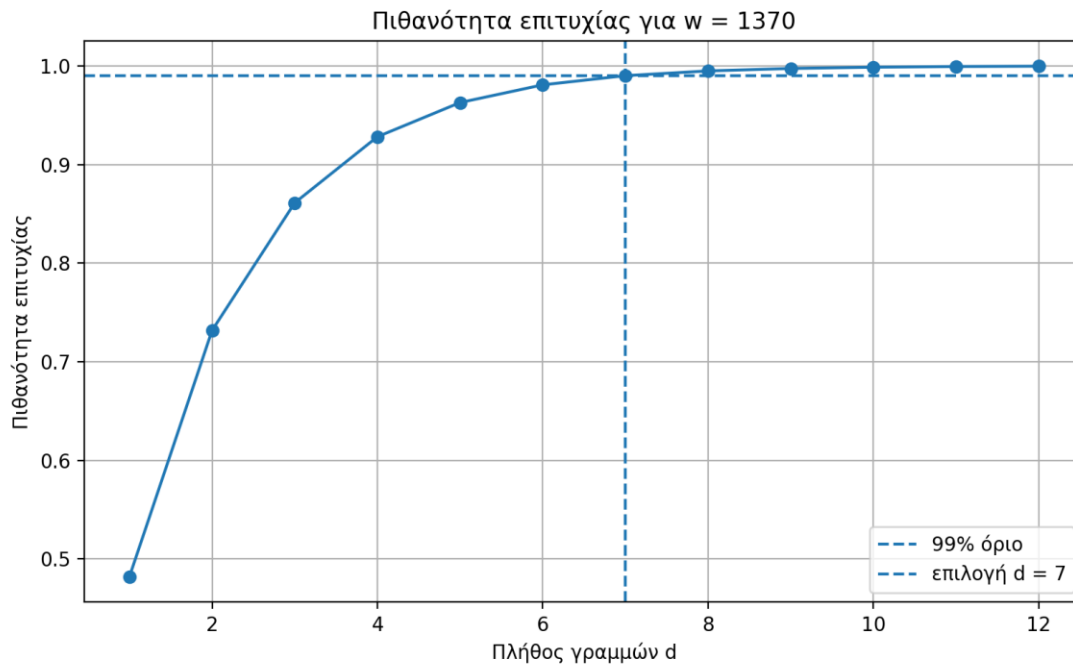
Η υλοποίηση έγινε σε Python. Τα 100-bit strings τα παριστάνω ως ακεραίους από 0 μέχρι $2^{100} - 1$, γιατί έτσι είναι πιο εύκολο να βρίσκω τα bits και να υπολογίζω τις hash functions. Η find_answer δεν κοιτάζει τη λίστα με τα τυχαία strings για να βρει τη σωστή απάντηση. Χρησιμοποιεί μόνο ό,τι έχει αποθηκευτεί μέσα στη δομή.

9. Ενδεικτικό πείραμα

Για να το δοκιμάσω, έφτιαξα 1000 διαφορετικές τυχαίες συμβολοσειρές των 100 bits. Παρατήρησα ότι οι πρώτες 999 μπαίνουν με τιμή 10 και η τελευταία με τιμή 100, όπως λέει η εκφώνηση. Έτρεξα το πείραμα 1000 φορές. Επειδή υπάρχει τυχειότητα, το ποσοστό δεν βγαίνει πάντα ακριβώς ίδιο. Μπορεί π.χ. να βγει 98.9%, 99.5%, 99.3%, 98.5% ή και 100%. Αυτό είναι φυσιολογικό και είναι κοντά στη θεωρητική τιμή. Στο verbose test τυπώνεται και το σωστό string και αυτό που επιστρέφει η δομή για να φανεί ότι όντως γίνεται ανακατασκευή και όχι απλά μέτρηση ποσοστού.

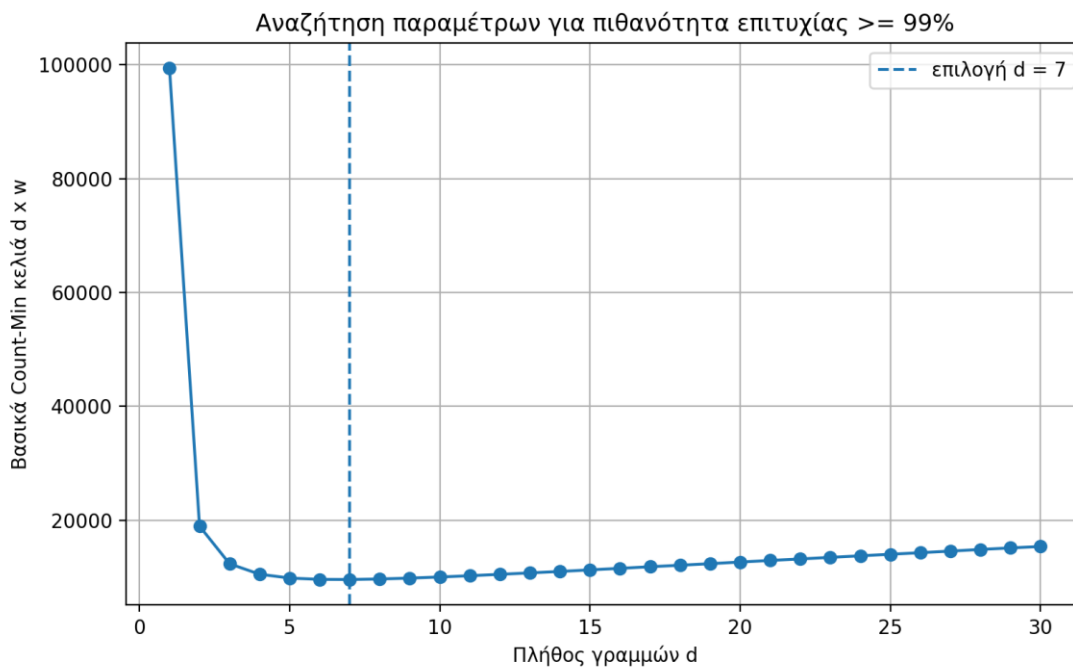
9.1 Γραφική απεικόνιση

Το πρώτο γράφημα δείχνει πως αλλάζει η πιθανότητα επιτυχίας όταν αλλάζει το d, κρατώντας σταθερό $w = 1370$.



Σχήμα 1: Πιθανότητα επιτυχίας για $w = 1370$ και διαφορετικές τιμές του d .

Το δεύτερο γράφημα δείχνει πόσα βασικά Count-Min buckets $d \cdot w$ χρειάζονται για διάφορες τιμές του d . Για κάθε d παίρνω το μικρότερο w που πιάνει πιθανότητα τουλάχιστον 99%.



Σχήμα 2: Βασικά Count-Min buckets $d*w$ για πιθανότητα επιτυχίας τουλάχιστον 99%.

Ενδεικτικές τιμές από την αναζήτηση:

d	w ελάχιστο	$d*w$
5	1969	9845
6	1602	9612
7	1370	9590
8	1210	9680
9	1093	9837
10	1003	10030
11	932	10252

10. Μνήμη

Η βασική Count-Min διάταξη έχει:

$$d*w = 7*1370 = 9590 \text{ buckets.}$$

Αυτός ο αριθμός είναι μόνο τα buckets και όχι όλοι οι μετρητές που κρατάει η επαυξημένη δομή. Σε αυτή τη λύση κάθε bucket χρειάζεται 1 μετρητή για το συνολικό άθροισμα και 100 επιπλέον μετρητές για τα bit-sums. Άρα, σε αναπαράσταση η λογική μνήμη είναι:

$$d*w*(100 + 1) = 968590 \text{ ακέραιοι μετρητές.}$$

Στον κώδικα το `table[d][w]` είναι πυκνό ενώ τα `bit_sums` είναι sparse. Άρα στην πράξη δεν κρατάω 100 bit-sums για κάθε άδειο bucket. Παρ' όλα αυτά, στην αναφορά πρέπει να είναι ξεκάθαρο ότι τα 9590 είναι μόνο τα βασικά buckets και όχι η συνολική μνήμη.

11. Συμπέρασμα

Η λύση δουλεύει επειδή το string με βάρος 100 είναι αρκετά πιο μεγάλο από τα υπόλοιπα, που έχουν βάρος 10. Αν απομονωθεί σε μία γραμμή τότε το bucket του έχει `total = 100` και τα `bit_sums` δείχνουν κατευθείαν τα bits του. Με $d = 7$ και $w = 1370$, η πιθανότητα να γίνει αυτή η απομόνωση είναι περίπου 99%, με το μοντέλο των σχεδόν ομοιόμορφων bucket choices. Το τελικό `mod w` κάνει την ανάλυση λίγο πιο προσεγγιστικά αλλά η διαφορά είναι ελάχιστη για το p που χρησιμοποιώ. Επίσης αν ο αλγόριθμος επιστρέψει κάποιο bucket δεν μπορεί να είναι λάθος bucket. Η μόνη περίπτωση που δεν βρίσκει απάντηση είναι όταν η βαριά συμβολοσειρά δεν απομονώνεται σε καμία γραμμή.